DETAILED DESIGN DOCUMENT

REVA Internship Opportunities (RIO) | School of Computer Science and Engineering, REVA University, Bengaluru

Particulars	Details
Task	Task 4 (SL 4), Project Track: Detailed Design of the Module
Name of Student	R Sai Pranav
University Seat Number (USN)	R23EM069
Project Title	ForgeGuard: An AI-Based Document Forgery Detection and Verification System
Track	PS 109
Faculty Guide	Mala B M
Date of Submission	2 November 2026

1. Introduction

1.1 Purpose. The System Design Document fixed the architecture: four detector services behind one orchestrator, one API, one data model. This document goes a full level deeper, into the classes, algorithms, data structures, interfaces and error handling that implement that architecture — detailed enough to code directly, not just to diagram. Every service gets the same six-part treatment: component breakdown, data-flow diagram, algorithm detail, data structures, interface specification, and error handling.

1.2 Scope. Section 2 covers the assigned module, Document Tamper Detection and Localisation, at the level of the actual working prototype in `tamper_localisation/` — including excerpts of the real code, parameter counts and a test-to-requirement map. Sections 3–6 cover the remaining three services and the orchestrator at the same structural depth, in pseudocode, since they are designed but not yet implemented. Section 7 gives the full column-level data design. Section 8 gives the full request/response schema for every API route. Section 9 is the complete system sequence diagram with error paths. Section 10 is system-wide error handling. Section 11 is the full traceability table.

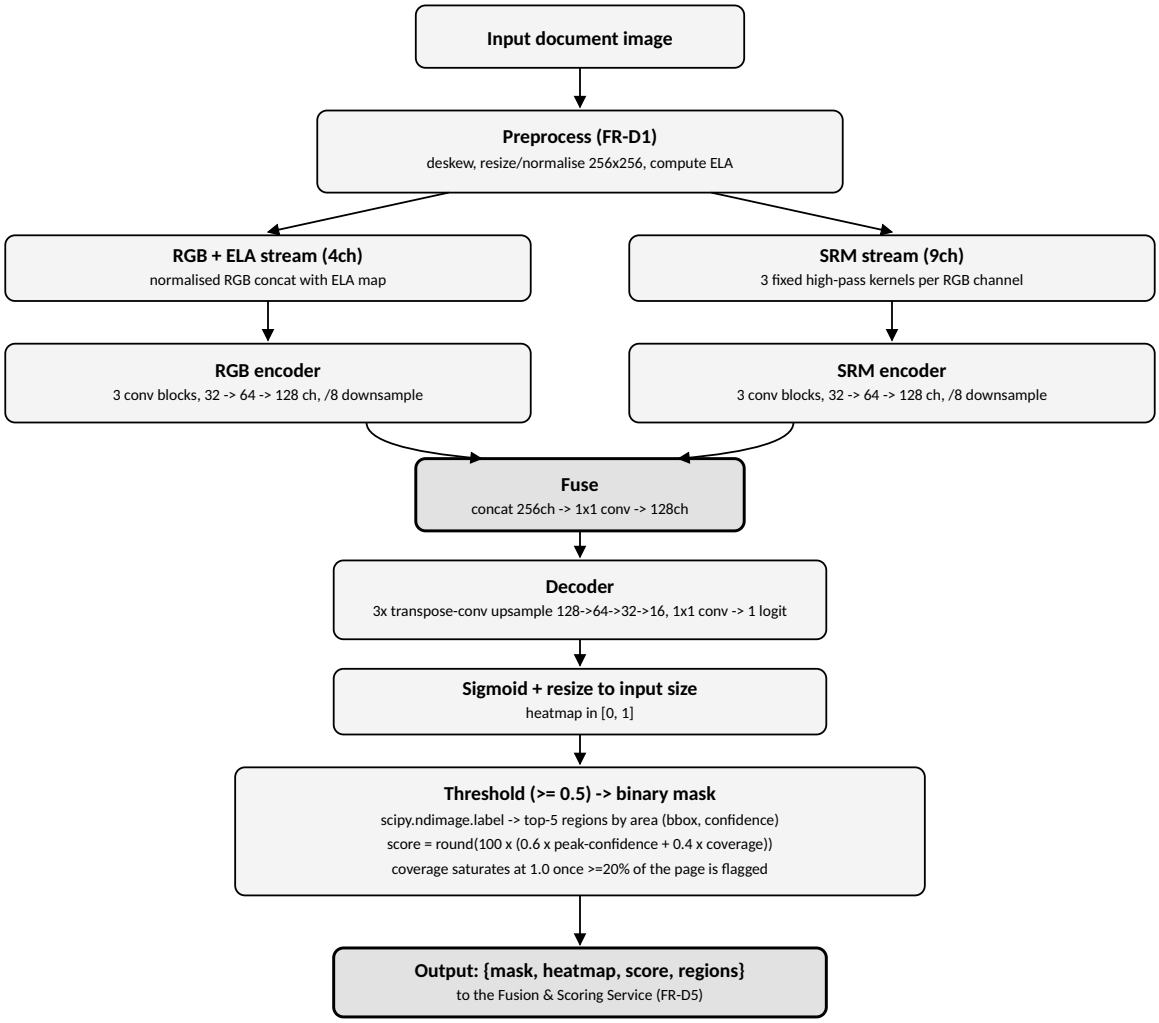
1.3 Identifiers. Gap N refers to the Literature Survey Report, O1–O9 to the Project Objectives, and FR-*/NFR-* to the System Design Document. All four documents trace against the same identifiers.

2. Detailed Design — Tamper Detection and Localisation (assigned module)

2.1 Component breakdown

Component	Responsibility
`SRMFilter`	Fixed, non-trainable depthwise convolution applying 3 high-pass kernels (horizontal residual, vertical residual, Laplacian-style) to each input channel. 3-channel RGB in → 9-channel noise-residual tensor out. Implements the noise-residual half of FR-D1/FR-D2.
`preprocess()`	FR-D1: deskew (row-variance search over ±10°), resize/normalise to 256×256 in [0,1], compute the ELA map (JPEG re-save at quality 90, pixel-wise difference). Returns the RGB tensor and the single-channel ELA map.
`TamperLocaliser` (model)	The dual-stream encoder-decoder itself: two independent encoders (RGB+ELA, SRM residual), a fusion layer, and a decoder that outputs a per-pixel tamper logit map. Pure `nn.Module`, no I/O.
`SyntheticTamperDataset`	Generates document-like images with a randomly spliced patch and a pixel-accurate mask, plus rotation/blur/noise/recompression augmentation (NFR-10). Stand-in for FSD (paper [29]) until real scans are available.
`TamperLocalisationService`	The module's only external surface (NFR-7): `predict(image)` runs preprocessing, the model, thresholding, connected-component region extraction and scoring, and returns the fixed output contract. Everything above is an implementation detail behind this one method.

2.2 Internal data flow



2.3 Algorithm detail

Deskew. For angle in [-10°, +10°] stepped by 1°, rotate the grayscale page and score it by the variance of row-wise pixel-intensity sums (text rows create high-variance peaks when horizontal; a skewed page blurs them together). Keep the angle with the highest variance. Cheap, needs no extra dependency, and good enough for the small scan tilts the problem calls for.

ELA. Re-save the page as JPEG at quality 90 and take the per-pixel absolute difference against the original, averaged over the 3 colour channels, normalised to [0, 1]. Untouched regions re-compress almost losslessly; edited regions (pasted from a different source, or re-saved at a different quality) show a brighter residual. Used only as one input channel alongside the RGB stream (Gap 4): the model is free to learn how much to trust it, rather than thresholding on it directly.

SRM kernels (applied per RGB channel, zero-padding, fixed weights, never updated by backprop):

```
horizontal residual   [[ 0,  0, 0], [0, -1, 1], [0,  0, 0]]
vertical residual     [[ 0,  0, 0], [0, -1, 0], [0,  1, 0]]
Laplacian residual    [[ 0,  1, 0], [1, -4, 1], [0,  1, 0]]
```

Loss. `BCEWithLogits(logits, mask) + DiceLoss(logits, mask)`, where `DiceLoss = 1 − (2|P∩T| + ε) / (|P| + |T| + ε)` on sigmoid probabilities `P` against ground truth `T`. BCE alone is dominated by the large background class on a sparsely-tampered page; Dice pulls the gradient toward getting the (small) tampered region right, which is what IoU in the NFR-5 evaluation actually measures.

Region extraction. Threshold the heatmap at 0.5, label connected components with `scipy.ndimage.label`, rank components by pixel area, keep the top 5, and report each as `{bbox: [x, y, w, h], confidence: mean heatmap value inside the component}` — this is the "largest suspicious regions with coordinates" FR-D6 asks for.

Tamper sub-score. `score = round(100 × (0.6 × peak_confidence + 0.4 × coverage_term))`, where `peak_confidence` is the heatmap's maximum value and `coverage_term = min(flagged_area_fraction / 0.2, 1.0)`. A small, high-confidence edit and a large, lower-confidence one can both reach a high score; a few stray low-confidence pixels cannot.

2.4 Reference implementation

The service method and its two private helpers, exactly as implemented in `tamper_localisation/infer.py`:

```
@torch.no_grad()
def predict(self, image: Image.Image, threshold: float = 0.5) -> dict:
    pre = preprocess(image, size=self.img_size)
    rgb = torch.from_numpy(pre["rgb"]).float().unsqueeze(0).to(self.device)
    ela =
torch.from_numpy(pre["ela"]).float().unsqueeze(0).unsqueeze(0).to(self.device)

    heatmap = torch.sigmoid(self.model(rgb, ela))[0, 0].cpu().numpy()  # (H, W) in
[0, 1]
    mask = (heatmap >= threshold).astype(np.uint8) * 255

    regions = self._top_regions(heatmap, threshold)
    score = self._tamper_score(heatmap, mask)
    return {"mask": mask, "heatmap": heatmap, "score": score, "regions": regions}

@staticmethod
def _tamper_score(heatmap: np.ndarray, mask: np.ndarray) -> int:
    """0-100: weighted blend of peak confidence and flagged-area coverage."""
    peak = float(heatmap.max())
    coverage = float((mask > 0).mean())
    coverage_term = min(coverage / 0.2, 1.0)
    return int(round(100 * (0.6 * peak + 0.4 * coverage_term)))
```

2.5 Data structures

RGB tensor	`torch.FloatTensor`, shape (3, 256, 256), values in [0, 1]
ELA map	`torch.FloatTensor`, shape (1, 256, 256), values in [0, 1]
SRM residual	`torch.FloatTensor`, shape (9, 256, 256), produced inside the model, never persisted
Logit map	`torch.FloatTensor`, shape (1, 256, 256), raw model output before sigmoid
Heatmap	`numpy.float32`, shape (256, 256), values in [0, 1]
Mask	`numpy.uint8`, shape (256, 256), values ∈ {0, 255}
Region	`{"bbox": [x, y, w, h] (int), "confidence": float}`, list ordered largest-area first, length ≤ 5
Score	`int`, 0–100

2.6 Interface specification (NFR-7)

Method	`TamperLocalisationService.predict(image: PIL.Image, threshold: float = 0.5) -> dict`
Input	`image`: a full page as a PIL image, any size. `threshold`: mask/region cutoff on the [0,1] heatmap.
Output	`mask`, `heatmap`, `score`, `regions` as defined in 2.5.
Construction	`TamperLocalisationService(checkpoint=None, device=None, img_size=256)`. A missing checkpoint runs the (untrained) architecture — useful for interface/contract tests, never for a real score.
Replaceability	The orchestrator only ever calls `predict()`. A newer localiser (different backbone, added anomaly head for Gap 8/O8) is a drop-in replacement for this class as long as it keeps the same method signature and output keys.

2.7 Complexity and performance (NFR-2)

Trainable parameters	~1.1M (two encoders at 32/64/128 channels, 1x1 fusion, 3-stage transpose-conv decoder) — roughly two orders of magnitude below a ResNet-50 backbone.
Fixed parameters	27 (the 3 SRM kernels x 3x3, never updated)
Forward pass cost	Dominated by the two /8-downsampling encoders over a 256x256 input; comparable to running two small CNNs over a quarter-megapixel image, well within the single-consumer-GPU-or-CPU budget NFR-2 sets.

Memory	Largest activation is the SRM stream at full input resolution (9, 256, 256) before the first downsample; no activation ever exceeds a few tens of MB at batch size 1.

2.8 Error handling and edge cases

- **No tamper present.** `regions` is an empty list and `score` is low; this is a normal, expected output, not an error.
- **Degenerate AUC.** If a batch's ground truth is all-genuine or all-forged, `roc_auc_score` is undefined; `evaluate.py` reports `nan` for that batch rather than raising, so a batch imbalance does not crash an evaluation run (NFR-5).
- **Untrained / missing checkpoint.** `TamperLocalisationService` runs with random weights if no checkpoint is given; callers must not treat that output as a real decision (see 2.6, Construction).
- **Device fallback.** `device` defaults to CUDA if available, else CPU, satisfying NFR-2 without any caller-side branching.
- **Oversized/undersized pages.** `preprocess()` resizes to a fixed 256×256 before the model ever sees the image, so input resolution cannot change the model's memory footprint or runtime.
- **Corrupt/unreadable image.** Image decoding happens before `predict()` is called (at the API gateway); the service assumes a valid, already-decoded `PIL.Image` and is not responsible for file-format errors.

2.9 Verification against `test_module.py`

Test	Checks
`test_model_forward_shape`	Model output shape matches input spatial size regardless of batch size — the decoder's upsampling and the final bilinear resize are correct (2.2).
`test_dataset_sample_shapes`	Synthetic dataset emits the exact tensor shapes/types the model consumes (2.5), and masks are binary.
`test_service_predict_contract`	End-to-end: `predict()` on an untrained model still satisfies the full output contract (2.6) — shapes, score range, region count.
`test_pixel_metrics_known_values`	Precision/recall/IoU formulas (NFR-5) against a hand-computed 2x2 example, independent of the model.

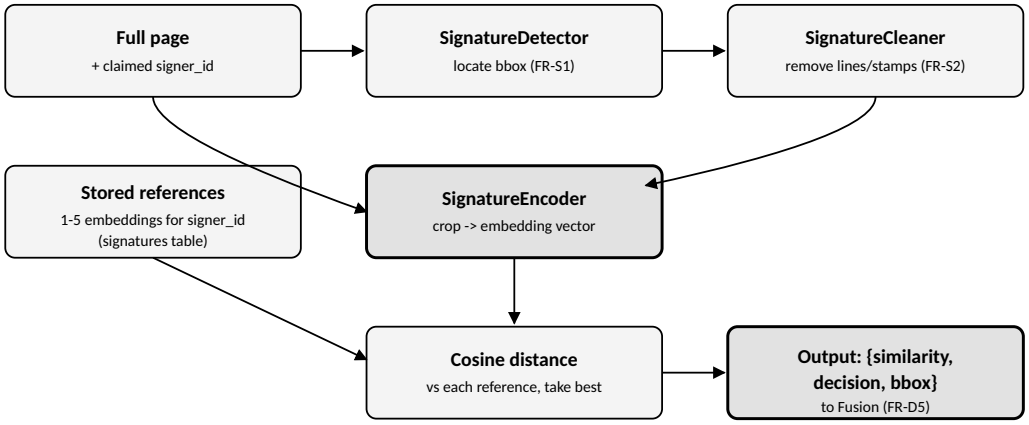
3. Detailed Design — Signature Verification Service

3.1 Component breakdown

Component	Responsibility

`SignatureDetector`	Object detector that locates the signature region on the full page and returns a bounding box (paper [13]). FR-S1.
`SignatureCleaner`	Removes ruled lines, stamps and background noise from the detected crop before embedding. FR-S2.
`SignatureEncoder`	Writer-independent Siamese/metric-learning encoder (papers [3], [6], [12]), frozen at inference; maps a signature crop to a fixed-length embedding vector.
`SignatureService`	The module's external surface: `verify(page, signer_id)` runs detect → clean → embed → compare and returns a similarity score and decision. FR-S3.

3.2 Data flow



3.3 Algorithm detail

Detection. A lightweight object detector (anchor-free, single class "signature") trained on signature bounding boxes over real document pages, following the multilingual detection approach of paper [13]; returns the highest-confidence box above a detection threshold, or "not found" if none clears it.

Cleaning. Binarise the crop, remove long straight lines (ruled lines under the signature) via morphological opening with a wide horizontal structuring element, and remove small disconnected blobs below an area threshold (stamp fragments, noise), following the cleaning stage of paper [13].

Comparison. `similarity = max(cosine(embed(crop), ref_i)) for ref_i in references`; `decision = "genuine" if similarity >= threshold else "forged"`. The threshold is tuned on a validation split, not hard-coded, so it can be recalibrated per deployment (NFR-5).

3.4 Data structures

Embedding	Fixed-length float vector (encoder output dimension, e.g. 128), L2-normalised before cosine comparison
Reference set	List of 1–5 embeddings per `signer_id`, loaded from the `signatures` table
Output	`{"similarity": float [0,1], "decision": "genuine"|"forged", "bbox": [x,y,w,h]}`

3.5 Interface specification (NFR-6, NFR-7)

Method	`SignatureService.verify(page: Image, signer_id: str) -> dict`
Replaceability	Adding a signer is an `INSERT` into `signatures`, never a retrain (NFR-6); swapping the encoder for a better metric-learning model changes nothing outside this class (NFR-7).

3.6 Error handling

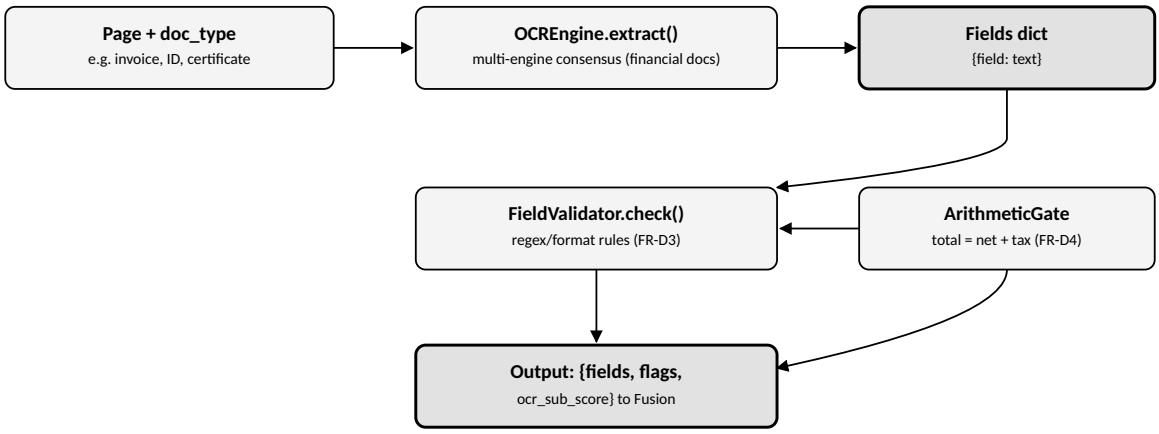
- **No signature detected.** Returns `decision = "not found"` rather than guessing; the fusion stage treats this sub-score as unavailable (Section 6.4).
- **No reference signatures registered for the signer.** Returns an error the orchestrator surfaces to the verifier (FR-U4 requires at least one reference before verification can run).

4. Detailed Design — OCR / Field Validation Service

4.1 Component breakdown

Component	Responsibility
`OCREngine`	Text extraction; consensus across multiple engines for financial documents, following FORENSIQ's triple-OCR approach (paper [39]).
`FieldValidator`	Per-document-type regex/format rules on names, dates and ID numbers. FR-D3.
`ArithmeticGate`	For financial documents, checks that the total equals the net amount plus tax. FR-D4.
`OCRService`	External surface: `check(page, doc_type) -> dict`, running extraction, field checks and (where applicable) the arithmetic gate.

4.2 Data flow



4.3 Algorithm detail

Extraction. For financial documents, run 3 independent OCR engines and take the field value a majority agree on (falling back to the highest-confidence single result if no majority), following FORENSIQ (paper [39]), which found this materially more reliable than any single engine. Non-financial documents use one engine.

Field validation. Each `doc_type` has a fixed set of regex/format rules (date formats, ID-number patterns, name character sets); a field fails if it does not match its rule, is empty where required, or (for a date) falls outside a sane range.

Arithmetic gate. `abs(total - (net + tax)) <= tolerance` (a small absolute/relative tolerance for rounding), applied only when `doc_type` is financial (invoice, receipt, bank statement).

OCR sub-score. `100 × (fraction of fields that passed) × (1 if arithmetic gate passes or is not applicable else 0.5)` — an arithmetic failure is weighted as a strong signal without zeroing the whole sub-score outright, since a single mis-OCR'd digit can also fail the gate on an otherwise genuine document.

4.4 Data structures

Fields	`dict[str, str]`, keyed by field name (e.g. `"date"`, `"total"`, `"id_number"`)
Flags	`dict[str, bool]`, one entry per field, `True` = passed its rule
Output	`{"fields": dict, "flags": dict, "arithmetic_ok": bool|None, "ocr_sub_score": int}`

4.5 Interface specification

Method	`OCRService.check(page: Image, doc_type: str) -> dict` (FR-D3, FR-D4)

4.6 Error handling

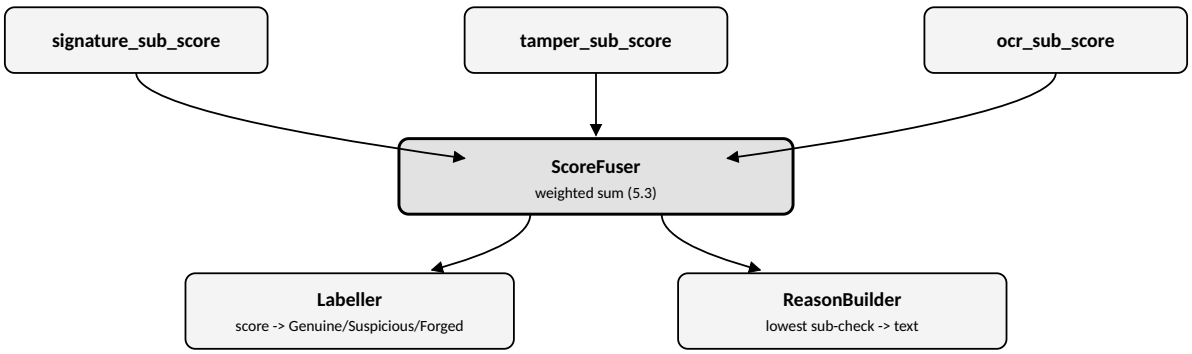
- **Unreadable text region.** A field the OCR engine cannot read at all is recorded as failed, not silently omitted, so the sub-score reflects the gap.
- **`doc_type` with no registered rule set.** Field validation is skipped and the OCR sub-score is derived from extraction confidence only; this is logged so new document types get rule coverage added, not silently scored as perfect.

5. Detailed Design — Fusion & Scoring Service

5.1 Component breakdown

Component	Responsibility
`ScoreFuser`	Combines the three sub-scores into one weighted authenticity score. FR-D5.
`Labeller`	Maps the fused score to Genuine/Suspicious/Forged against configurable thresholds.
`ReasonBuilder`	Builds the plain-language explanation from the lowest-scoring sub-check. FR-D6.
`FusionService`	External surface: `score(signature, tamper, ocr) -> dict`.

5.2 Data flow



5.3 Algorithm detail

```
final_score = w_sig * signature_sub_score
            + w_tmp * tamper_sub_score
            + w_ocr * ocr_sub_score          # w_sig + w_tmp + w_ocr = 1, default
1/3 each

label = "Genuine"     if final_score >= 80
      = "Suspicious"   if 40 <= final_score < 80
      = "Forged"       if final_score < 40

reason = template(lowest-scoring sub-check, its threshold, its evidence)
       # e.g. "signature similarity 0.41 (below 0.6 threshold)"
```

A weighted rule, not a learned end-to-end classifier (Gap 5/O5): each sub-score and the reason it produced stay inspectable, which a black-box classifier would not give (Gap 6/O6). Weights and thresholds are configuration, not code, so they can be recalibrated per deployment or document type without a redeploy.

5.4 Data structures

Weights	`{w_sig: float, w_tmp: float, w_ocr: float}`, sum to 1, stored as configuration
Thresholds	`{genuine: 80, suspicious_floor: 40}`, configuration
Output	`{"score": int, "label": str, "reason": str, "sub_scores": {signature, tamper, ocr}, "heatmap_key": str}` — this is the row persisted into `verifications` (Section 7)

5.5 Interface specification

Method	`FusionService.score(signature: dict, tamper: dict, ocr: dict) -> dict` (FR-D5, FR-D6)

5.6 Error handling

- **A sub-score is missing** (detector failed or timed out, Section 6.4). Its weight is redistributed proportionally across the remaining sub-scores, and the label is capped at `Suspicious` — a verification is never reported `Genuine` on partial evidence.
- **All three sub-scores missing.** The request itself is reported as failed rather than fused into a meaningless label.

6. Detailed Design — Verification Orchestrator

6.1 Responsibilities

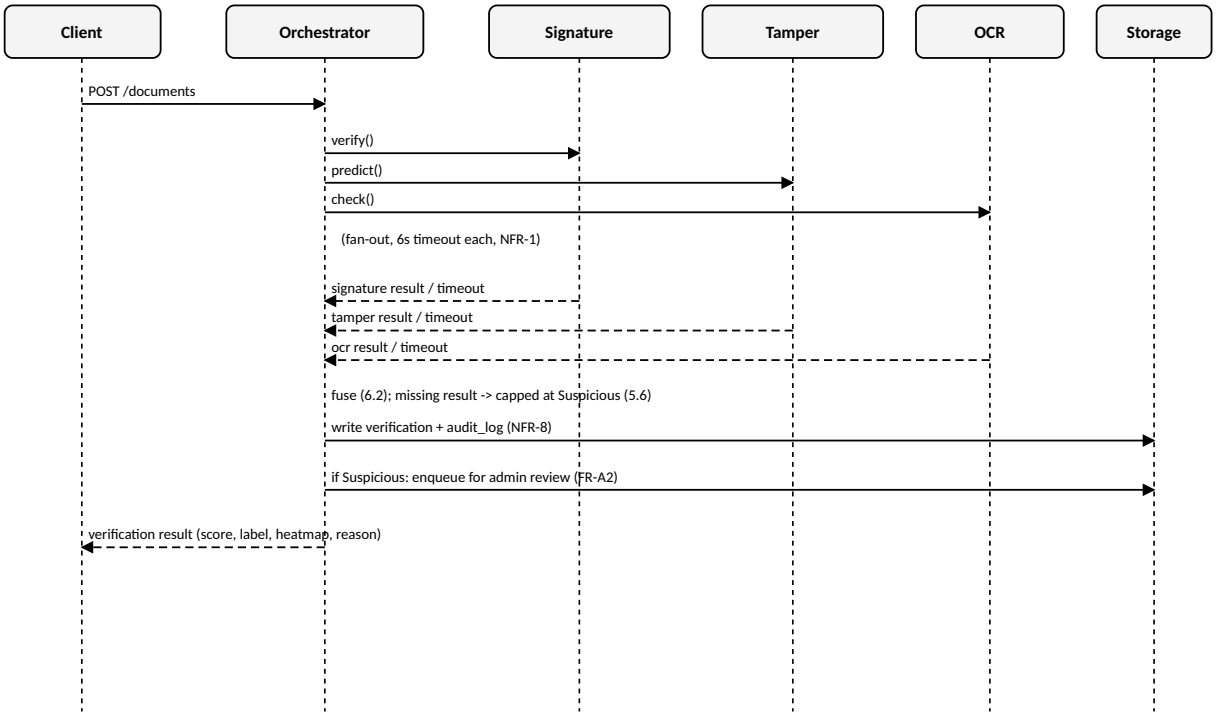
The orchestrator is the single entry point (FR-P1) for every verification: it pre-processes the upload, fans the three detector calls out, fans their results back in, calls fusion, writes the audit log, and returns to the caller. It contains no detection logic of its own.

6.2 Dispatch algorithm

```
def verify(document):
    pre = preprocess(document)                          # FR-D1, shared pre-pass
    tasks = {
        "signature": call_with_timeout(signature_service.verify, pre, timeout=6),
        "tamper":    call_with_timeout(tamper_service.predict,  pre, timeout=6),
        "ocr":       call_with_timeout(ocr_service.check,       pre, timeout=6),
    }
    results = await_all(tasks)          # fan-in; NFR-1 budget: 10s total per page
    for name, outcome in results.items():
        if outcome is TIMEOUT or outcome is ERROR:
            results[name] = None        # treated as missing evidence, not a crash

    fused = fusion_service.score(results["signature"], results["tamper"],
results["ocr"])
    audit_log.write(document.id, fused)              # NFR-8, append-only
    if fused.label == "Suspicious":
        admin_queue.add(document.id)                  # FR-A2
    return fused
```

6.3 Full system sequence



6.4 Concurrency and failure handling

- Each detector call runs under its own timeout (6s default, leaving headroom inside the 10s NFR-1 budget for pre-processing and fusion).
- A timeout or exception in one detector does not cancel the others; the orchestrator awaits whichever complete and records `None` for the rest.
- Fusion (5.6) is the single place that decides how to degrade a label when evidence is missing — the orchestrator itself makes no scoring decisions.
- Audit log writes (Section 7) are the last step before returning, so a result is never handed to the client without a corresponding record.

7. Detailed Data Design

Table	Columns (type, constraint)
`users`	`id` UUID PK · `email` text UNIQUE NOT NULL · `password_hash` text NOT NULL · `role` enum(verifier, admin) NOT NULL · `created_at` timestamptz NOT NULL
`signatures`	`id` UUID PK · `user_id` UUID FK→users · `image_key` text NOT NULL · `embedding` float[] NOT NULL · `created_at` timestamptz NOT NULL · CHECK: ≤5 rows per `user_id` (FR-U4)

documents	id UUID PK · uploaded_by UUID FK→users · doc_type text NOT NULL · file_key text NOT NULL · sha256_hash char(64) NOT NULL, INDEXED (FR-P2) · uploaded_at timestamptz NOT NULL
verifications	id UUID PK · document_id UUID FK→documents · signature_sub_score int NULL · tamper_sub_score int NULL · ocr_sub_score int NULL · final_score int NOT NULL · label enum(Genuine, Suspicious, Forged) NOT NULL · heatmap_key text NULL · reason_text text NOT NULL · created_at timestamptz NOT NULL, INDEXED
audit_log	id UUID PK · actor_id UUID FK→users NULL (system actions) · action text NOT NULL · verification_id UUID FK→verifications NULL · detail_json jsonb NOT NULL · created_at timestamptz NOT NULL · no UPDATE/DELETE grant for the application role (NFR-8)
feedback	id UUID PK · verification_id UUID FK→verifications · admin_id UUID FK→users · final_decision enum(Genuine, Forged) NOT NULL · created_at timestamptz NOT NULL · append-only, latest row per verification_id is authoritative (FR-A2)

8. Detailed API Design

Route	Request / Response
POST /auth/register	req {email, password} → 201 {id, email, role} / 409 if email taken
POST /auth/login	req {email, password} → 200 {token} / 401 on bad credentials
POST /signatures	req multipart {signer_id, image} → 201 {id} / 400 if signer already has 5 references (FR-U4)
POST /documents	req multipart {file, doc_type} → 202 {document_id} / 400 unsupported file type / 413 too large
GET /documents/{id}/verification	200 {score, label, heatmap_url, sub_scores, reason} / 202 if still processing / 404
GET /documents/{id}/report.pdf	200 PDF stream / 404
GET /verifications?user_id=	200 [{id, document_id, score, label, created_at}], paginated

POST /admin/verifications/{id}/decision	req `{final_decision}` → 201 (FR-A2) / 403 if role != admin (NFR-9)
GET /admin/audit-log	200 `[{action, actor_id, verification_id, detail, created_at}]`, paginated / 403
GET /admin/analytics	200 `{counts_by_label, avg_score, avg_latency}` / 403

9. System-Wide Error Handling

Invalid file / type	400 at the gateway, before the orchestrator; never coerced into a wrong document type
Detector timeout/failure	Handled per Section 6.4: request completes, capped confidence, reason notes the gap — never a 500
Auth/role failure	401/403 at the gateway (NFR-9); not logged as a verification attempt
Concurrent admin decisions	`feedback` append-only; latest row authoritative, all prior rows retained (NFR-8)
Storage write failure	Verification is not returned to the client as complete until its `audit_log` row is confirmed written

10. Traceability

Section	Traces to
2 (assigned module)	O1–O4, O6–O8; FR-D1, FR-D2, FR-D6; NFR-2, NFR-5, NFR-7, NFR-10; Gap 3, Gap 4
3 (signature)	O1, O2; FR-S1–S3; NFR-6; Gap 1, Gap 2
4 (OCR/field)	FR-D3, FR-D4; Gap 5
5 (fusion)	O5, O6; FR-D5, FR-D6; Gap 5, Gap 6
6 (orchestrator)	FR-P1; NFR-1
7 (data design)	FR-P2, FR-A2, FR-A3, FR-U4; NFR-8
8 (API design)	FR-P1, FR-U1–U4, FR-A1–A3
9 (error handling)	NFR-1, NFR-8, NFR-9

11. Conclusion

The assigned module is detailed to the level of running code: the component breakdown, data flow diagram, algorithm descriptions and reference implementation in Section 2 match `tamper_localisation/` line for line, and Section 2.9 ties each self-check in `test_module.py` back to the requirement it verifies. The remaining services, the orchestrator, the data design and the API are detailed to the same structural depth — component, data flow, algorithm, data structures, interface, error handling — so implementing them means filling in the classes and functions named here against contracts that are already fixed.